

運用 PaLM 和 LangChain4j，以 Java 建立生成式 AI 產生文字

來源：<https://codelabs.developers.google.com/codelabs/genai-text-gen-java-palm-langchain4j?hl=zh-tw>

步驟數：8

在這個程式碼研究室中，您將開始在 Java 中使用生成式 AI、整合 PaLM 大型語言模型，並運用 LangChain4J LLM 自動化調度管理架構

目錄

1. [1. 簡介](#)
2. [2. 設定和需求](#)
3. [3. 準備開發環境](#)
4. [4. 首次呼叫 PaLM 的文字模型](#)
5. [5. 從非結構化文字中擷取資訊](#)
6. [6. 提示範本和結構化提示](#)
7. [7. 分類文字和分析情緒](#)
8. [8. 恭喜](#)

步驟 1 · 約 0 分鐘

1. 簡介

上次更新時間：2023 年 11 月 27 日

什麼是生成式 AI

生成式 AI 又稱為「生成式人工智慧」，是指使用 AI 技術生成新的文字、圖片、音樂、音訊和影片等內容。

生成式 AI 是由基礎模型 (大型 AI 模型) 驅動，可同時處理多項工作，並立即執行各種任務，包括提供摘要、問與答和分類等。此外，只需要少量訓練，就能以極少的範例資料針對指定用途調整基礎模型。

生成式 AI 的運作方式為何？

生成式 AI 的運作方式是使用機器學習模型，根據一系列人類創作內容的資料來掌握相關模式與關係，再運用學到的模式生成新內容。

「監督式學習」是最常用來訓練生成式 AI 模型的方法，也就是向模型提供一系列人類創作內容和相應的標籤，讓模型學習生成與人類創作內容相似的內容，並加上相同的標籤。

常見的生成式 AI 應用程式有哪些？

生成式 AI 可以處理大量內容，透過文字、圖片和容易使用的格式，產生相關洞察資訊和答案。生成式 AI 有助於：

- 增強即時通訊與搜尋體驗，藉此改善與顧客的互動方式
- 運用對話式介面和摘要功能，探索大量非結構化資料
- 協助處理重複性工作，例如回覆提案請求 (RFP)、以五種語言將行銷內容本地化，以及確認客戶合約是否符合相關法規等

Google Cloud 提供哪些生成式 AI 產品？

[Vertex AI](#) 能讓您將自訂的基礎模型嵌入應用程式中，並與應用程式互動，而且不需專業的機器學習知識，就能輕鬆上手。您可以在 [Model Garden](#) 中存取基礎模型、透過 [Generative AI Studio](#) 的簡易使用者介面調整模型，或是運用數據資料學筆記本中的模型。

有了 [Vertex AI Search and Conversation](#)，開發人員就能在最短時間內，建構生成式 AI 技術輔助[搜尋引擎](#)和聊天機器人。

此外，[Duet AI](#) 是 AI 技術輔助協作工具，在 Google Cloud 和 IDE 中皆可使用，能協助您以更快的速度完成更多工作。

本程式碼研究室的重點為何？

本程式碼研究室著重於 [PaLM 2](#) 大型語言模型 (LLM)，該模型託管於 Google Cloud Vertex AI，涵蓋所有機器學習產品和服務。

您將使用 Java 與 PaLM API 互動，並搭配 [LangChain4J](#) LLM 架構協調器。您將透過各種具體範例，瞭解如何運用 LLM 回答問題、產生構想、擷取實體和結構化內容，以及生成摘要。

請進一步說明 LangChain4J 架構！

[LangChain4J](#) 框架是開放原始碼程式庫，可透過自動調度管理各種元件 (例如 LLM 本身，以及向量資料庫 (用於語意搜尋)、文件載入器和分割器 (用於分析文件並從中學習)、輸出剖析器等其他工具)，將大型語言模型整合至 Java 應用程式。

從 [Github 專案頁面](#)：

這個專案的目標是簡化將 AI/LLM 功能整合至 Java 應用程式的程序。

這要歸功於：

- **簡單且連貫的抽象層**，可確保程式碼不會依附於具體實作內容，例如 LLM 供應商、嵌入式儲存空間供應商等。這樣一來，您就能輕鬆更換元件。
- **上述抽象概念的眾多實作項目**，提供多種 LLM 和嵌入式儲存空間供您選擇。
- **熱門功能**，例如：
 - **匯入自有資料** (文件、程式碼集等)，讓 LLM 根據您的資料採取行動及回覆。
 - **自主代理**：將 (即時定義的) 工作委派給 LLM，並盡力完成工作。
 - **提示範本**：協助您盡可能提高 LLM 回覆的品質。
 - **記憶功能**可為 LLM 提供當前和先前的對話脈絡。
 - **結構化輸出內容**：以 Java POJO 形式接收 LLM 回覆，並取得所需結構。
 - **「AI 服務」**：用於以宣告方式定義簡單 API 背後複雜的 AI 行為。
 - **鏈結**：減少常見用途中大量樣板程式碼的需求。
 - **自動審查**，確保 LLM 的所有輸入和輸出內容都不會造成危害。



課程內容

- 如何設定 Java 專案，以便使用 PaLM 和 LangChain4J
- 如何首次呼叫 PaLM 文字模型，生成內容及回答問題

- 如何從非結構化內容擷取實用資訊 (實體或關鍵字擷取，以 JSON 格式輸出)
- 如何使用少量樣本提示進行內容分類或情緒分析

軟硬體需求

- 熟悉 Java 程式設計語言
 - 具備 Google Cloud 專案
 - 瀏覽器，例如 Chrome 或 Firefox
-

2. 設定和需求

自修實驗室環境設定

1. 登入 [Google Cloud 控制台](#)，然後建立新專案或重複使用現有專案。如果沒有 Gmail 或 Google Workspace 帳戶，請先[建立帳戶](#)。

The image shows two screenshots from the Google Cloud console. The top screenshot shows the 'Select a project' dropdown menu in the top navigation bar, with a blue arrow pointing to it. The bottom screenshot shows the 'New Project' form. The 'Project name' field contains 'My Project 13475'. The 'Project ID' field contains 'inbound-analogy-389614', which is highlighted with a blue box. The 'Location' field is empty, and the 'BROWSE' button is visible. The 'CREATE' and 'CANCEL' buttons are at the bottom.

- **專案名稱**是這個專案參與者的顯示名稱。這是 Google API 未使用的字元字串。你隨時可以更新。
- **專案 ID** 在所有 Google Cloud 專案中都是不重複的，而且設定後即無法變更。Cloud 控制台會自動產生專屬字串，通常您不需要在意該字串為何。在大多數程式碼研究室中，您需要參照專案 ID (通常標示為 `PROJECT_ID`)。如果您不喜歡產生的 ID，可以產生另一個隨機 ID。你也可以嘗試使用自己的名稱，看看是否可用。完成這個步驟後就無法變更，且專案期間會維持不變。
- 請注意，有些 API 會使用第三個值，也就是「專案編號」。如要進一步瞭解這三種值，請參閱[說明文件](#)。

注意：專案 ID 在全域中是唯一的，選定後就無法再供他人使用。您是該 ID 的唯一使用者。即使專案已刪除，ID 也無法再次使用

注意：如果使用 Gmail 帳戶，可以將預設位置設為「沒有機構」。如果你使用 Google Workspace 帳戶，請選擇適合貴機構的位置。

2. 接著，您需要在 Cloud 控制台中[啟用帳單](#)，才能使用 Cloud 資源/API。完成這個程式碼研究室的費用不高，甚至可能完全免費。如要關閉資源，避免在本教學課程結束後繼續產生費用，請刪除您建立的資源或專案。Google Cloud 新使

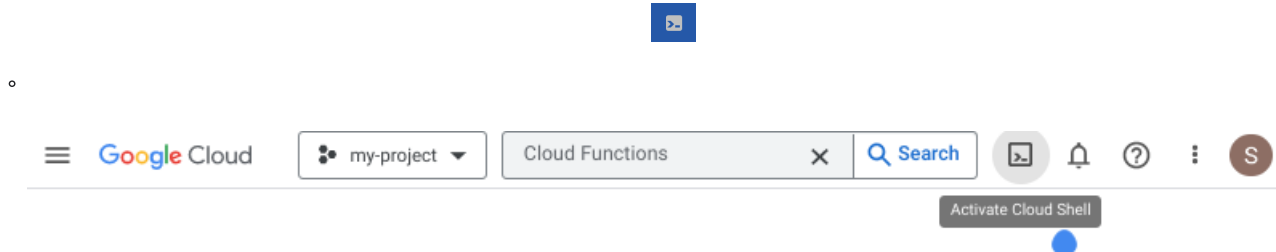
用者可參加[價值\\$300 美元的免費試用計畫](#)。

啟動 Cloud Shell

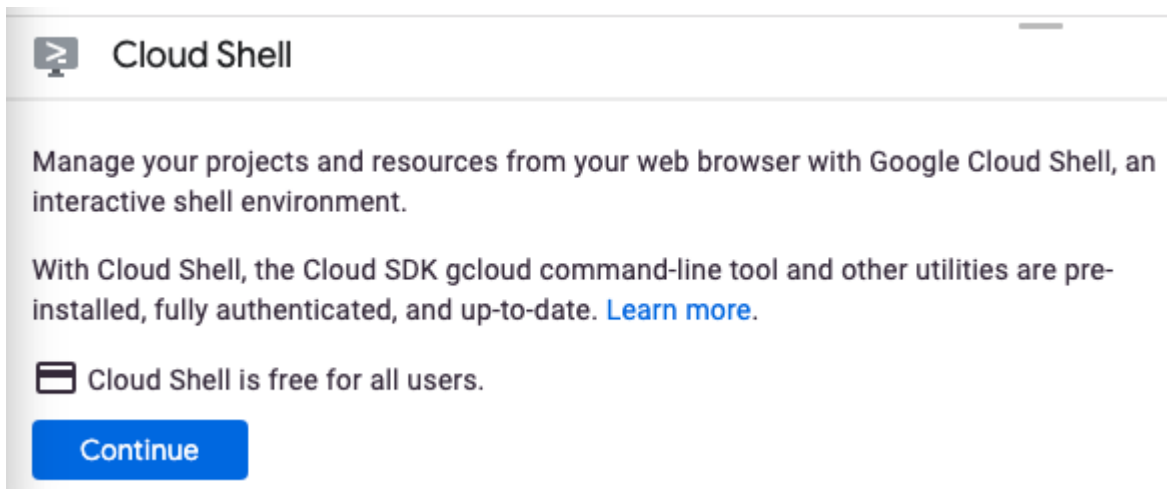
雖然您可以透過筆電遠端操作 Google Cloud，但在本程式碼研究室中，您將使用 [Cloud Shell](#)，這是 Cloud 中執行的指令列環境。

啟用 Cloud Shell

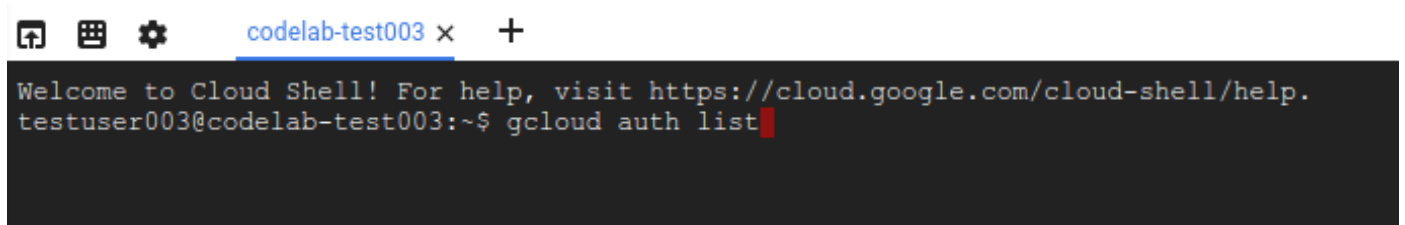
1. 在 Cloud 控制台，點選「啟用 Cloud Shell」圖示



如果您是首次啟動 Cloud Shell，系統會顯示中繼畫面，說明這個指令列環境。如果出現中繼畫面，請按一下「繼續」。



佈建並連至 Cloud Shell 預計只需要幾分鐘。



這部虛擬機器已載入所有必要的開發工具，並提供永久的 5 GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。本程式碼研究室幾乎所有工作都可在瀏覽器上完成。

連至 Cloud Shell 後，您應該會看到驗證已完成，專案也已設為獲派的專案 ID。

2. 在 Cloud Shell 中執行下列指令，確認您已通過驗證：

```
gcloud auth list
```

指令輸出

Credentialed Accounts

ACTIVE ACCOUNT

* <my_account>@<my_domain.com>

To set the active account, run:

```
$ gcloud config set account `ACCOUNT`
```

注意： `gcloud` 指令列工具是 Google Cloud 中功能強大的整合式指令列工具，並已預先安裝於 Cloud Shell。你會發現它支援 Tab 鍵自動完成功能。第一次執行指令時，系統可能會提示您進行驗證。詳情請參閱 [gcloud 指令列工具總覽](#)。

3. 在 Cloud Shell 中執行下列指令，確認 `gcloud` 指令知道您的專案：

```
gcloud config list project
```

指令輸出

```
[core]
project = <PROJECT_ID>
```

如未設定，請輸入下列指令手動設定專案：

```
gcloud config set project <PROJECT_ID>
```

指令輸出

```
Updated property [core/project].
```

注意： 如果您在 Cloud Shell 以外的環境完成本教學課程，請按照「[設定應用程式預設憑證](#)」一文的說明操作。

步驟 3 · 約 6 分鐘

3. 準備開發環境

在本程式碼研究室中，您將使用 Cloud Shell 終端機和程式碼編輯器開發 Java 程式。

啟用 Vertex AI API

1. 在 [Google Cloud 控制台](#) 中，確認專案名稱顯示在頂端。如果不是，請點按「選取專案」開啟**專案選取器**，然後選取所需專案。
2. 如果不在 Google Cloud 控制台的 Vertex AI 部分，請執行下列操作：
3. 在「搜尋」中輸入「Vertex AI」，然後按下 Enter 鍵
4. 在搜尋結果中，按一下「Vertex AI」，系統會顯示 Vertex AI 資訊主頁。
5. 在 Vertex AI 資訊主頁中，點按「啟用所有建議的 API」。

資訊：啟用程序可能需要一些時間才能完成。啟用 API 時，Google Cloud 控制台右上方的鈴鐺圖示會顯示藍色圓圈。

這會啟用多個 API，但本程式碼研究室最重要的 API 是 `aiplatform.googleapis.com`。您也可以在 Cloud Shell 終端機中執行下列指令，透過指令列啟用該 API：

```
$ gcloud services enable aiplatform.googleapis.com
```

使用 Gradle 建立專案結構

如要建構 Java 程式碼範例，您將使用 [Gradle](#) 建構工具和 Java 17 版。如要使用 Gradle 設定專案，請在 Cloud Shell 終端機中建立目錄 (這裡為 `palm-workshop`)，然後在該目錄中執行 `gradle init` 指令：

```
$ mkdir palm-workshop
$ cd palm-workshop

$ gradle init

Select type of project to generate:
  1: basic
  2: application
  3: library
  4: Gradle plugin
Enter selection (default: basic) [1..4] 2

Select implementation language:
  1: C++
  2: Groovy
  3: Java
  4: Kotlin
  5: Scala
  6: Swift
Enter selection (default: Java) [1..6] 3

Split functionality across multiple subprojects?:
  1: no - only one application project
  2: yes - application and library projects
Enter selection (default: no - only one application project) [1..2] 1

Select build script DSL:
  1: Groovy
  2: Kotlin
Enter selection (default: Groovy) [1..2] 1

Generate build using new APIs and behavior (some features may change in the next
minor release)? (default: no) [yes, no]

Select test framework:
  1: JUnit 4
  2: TestNG
  3: Spock
  4: JUnit Jupiter
Enter selection (default: JUnit Jupiter) [1..4] 4

Project name (default: palm-workshop):
Source package (default: palm.workshop):

> Task :init

Get more help with your project:
https://docs.gradle.org/7.4/samples/sample\_building\_java\_applications.html
```

```
BUILD SUCCESSFUL in 51s
2 actionable tasks: 2 executed
```

您將使用 **Java 語言** (選項 3) 建構應用程式 (選項 2)，不使用子專案 (選項 1)，並使用建構檔案的 **Groovy 語法** (選項 1)，不使用新的建構功能 (選項 0)，使用 **JUnit Jupiter** (選項 4) 產生測試，專案名稱可使用 **palm-workshop**，來源套件則可使用 **palm.workshop**。

注意：如果您為專案選擇了不同的名稱和來源套件，請務必在程式碼研究室的後續步驟中使用這些名稱，因為操作說明會參照專案建立時選擇的名稱。

根據預設，專案名稱和套件名稱會從您初始化專案的存放區名稱推斷而來。對於 `"palm-workshop"` 目錄，建議使用 `"palm-workshop"` 做為專案名稱，並使用 `"palm.workshop"` 做為套件。

專案結構如下所示：

```
|— gradle
|   |— ...
|— gradlew
|— gradlew.bat
|— settings.gradle
|— app
|   |— build.gradle
|   |— src
|       |— main
|           |— java
|               |— palm
|                   |— workshop
|                       |— App.java
|— test
|   |— ...
```

現在來更新 `app/build.gradle` 檔案，加入一些必要的依附元件。如果存在 `guava` 依附元件，您可以移除該元件，並替換成 `LangChain4J` 專案的依附元件和記錄檔程式庫，以免出現缺少記錄器的訊息：

```
dependencies {
    // Use JUnit Jupiter for testing.
    testImplementation 'org.junit.jupiter:junit-jupiter:5.8.1'

    // Logging library
    implementation 'org.slf4j:slf4j-jdk14:2.0.9'

    // This dependency is used by the application.
    implementation 'dev.langchain4j:langchain4j-vertex-ai:0.24.0'
    implementation 'dev.langchain4j:langchain4j:0.24.0'
}
```

`LangChain4J` 有 2 個依附元件：

- 一個是核心專案，
- 以及專屬的 Vertex AI 模組。

如要使用 Java 17 編譯及執行程式，請在 `plugins {}` 區塊下方新增下列區塊：

```
java {  
    toolchain {  
        languageVersion = JavaLanguageVersion.of(17)  
    }  
}
```

警告：如果 Java 17 不是 Cloud Shell 環境中安裝的預設版本，請執行下列指令，將其設為預設版本：

```
$ sudo update-java-alternatives --set java-1.17.0-openjdk-amd64
```

還有一項變更要做：更新 `app/build.gradle` 的 `application` 區塊，讓使用者在叫用建構工具時，能夠覆寫要在指令列執行的主要類別：

```
application {  
    mainClass = providers.systemProperty('javaMainClass')  
                .orElse('palm.workshop.App')  
}
```

如要確認建構檔案是否已準備好執行應用程式，可以執行預設的主要類別，列印簡單的 `Hello World!` 訊息：

```
$ ./gradlew run -DjavaMainClass=palm.workshop.App  
  
> Task :app:run  
Hello World!  
  
BUILD SUCCESSFUL in 3s  
2 actionable tasks: 2 executed
```

現在您可以使用 LangChain4J 專案，透過 PaLM 大型語言文字模型進程式設計！

如需參考，以下是完整的 `app/build.gradle` 建構檔案現在應有的樣子：

```
plugins {
    // Apply the application plugin to add support for building a CLI application in Java.
    id 'application'
}

java {
    toolchain {
        // Ensure we compile and run on Java 17
        languageVersion = JavaLanguageVersion.of(17)
    }
}

repositories {
    // Use Maven Central for resolving dependencies.
    mavenCentral()
}

dependencies {
    // Use JUnit Jupiter for testing.
    testImplementation 'org.junit.jupiter:junit-jupiter:5.8.1'

    // This dependency is used by the application.
    implementation 'dev.langchain4j:langchain4j-vertex-ai:0.24.0'
    implementation 'dev.langchain4j:langchain4j:0.24.0'
    implementation 'org.slf4j:slf4j-jdk14:2.0.9'
}

application {
    mainClass = providers.systemProperty('javaMainClass').orElse('palm.workshop.App')
}

tasks.named('test') {
    // Use JUnit Platform for unit tests.
    useJUnitPlatform()
}
```

步驟 4 · 約 5 分鐘

4. 首次呼叫 PaLM 的文字模型

專案設定完成後，就可以呼叫 PaLM API 了。

在 `app/src/main/java/palm/workshop` 目錄中建立名為 `TextPrompts.java` 的新類別 (與預設的 `App.java` 類別並列)，然後輸入下列內容：

```
package palm.workshop;

import dev.langchain4j.model.output.Response;
import dev.langchain4j.model.vertexai.VertexAiLanguageModel;

public class TextPrompts {
    public static void main(String[] args) {
        VertexAiLanguageModel model = VertexAiLanguageModel.builder()
            .endpoint("us-central1-aiplatform.googleapis.com:443")
            .project("YOUR_PROJECT_ID")
            .location("us-central1")
            .publisher("google")
            .modelName("text-bison@001")
            .maxOutputTokens(500)
            .build();

        Response<String> response = model.generate("What are large language models?");

        System.out.println(response.content());
    }
}
```

重要事項：請務必變更專案 ID，使其與您自己的專案名稱相符！

在第一個範例中，您需要匯入 `Response` 類別，以及 PaLM 的 Vertex AI 語言模型。

接著，在 `main` 方法中，您將使用 `VertexAiLanguageModel` 的建構工具設定語言模型，指定：

- 端點，
- 專案，
- 區域，
- 發布商
- 以及模型名稱 (`text-bison@001`)。

語言模型準備就緒後，即可呼叫 `generate()` 方法並傳遞「提示」(即要傳送至 LLM 的問題或指令)。在這裡，你可以簡單詢問 LLM 是什麼。但你可以隨意變更提示，嘗試不同的問題或工作。

如要執行這個類別，請在 Cloud Shell 終端機中執行下列指令：

```
./gradlew run -DjavaMainClass=palm.workshop.TextPrompts
```

畫面會顯示類似如下的輸出：

Large language models (LLMs) are artificial intelligence systems that can understand and generate human language. They are trained on massive datasets of text and code, and can learn to perform a wide variety of tasks, such as translating languages, writing different kinds of creative content, and answering your questions in an informative way.

LLMs are still under development, but they have the potential to revolutionize many industries. For example, they could be used to create more accurate and personalized customer service experiences, to help doctors diagnose and treat diseases, and to develop new forms of creative expression.

However, LLMs also raise a number of ethical concerns. For example, they could be used to create fake news and propaganda, to manipulate people's behavior, and to invade people's privacy. It is important to carefully consider the potential risks and benefits of LLMs before they are widely used.

Here are some of the key features of LLMs:

- * They are trained on massive datasets of text and code.
- * They can learn to perform a wide variety of tasks, such as translating languages, writing different kinds of creative content, and answering your questions in an informative way.
- * They are still under development, but they have the potential to revolutionize many industries.
- * They raise a number of ethical concerns, such as the potential for fake news, propaganda, and invasion of privacy.

`VertexAILanguageModel` 建構工具可讓您定義選用參數，這些參數已有一些預設值，您可以覆寫這些值。例如：

- `.temperature(0.2)`：定義回覆的創意程度 (0 代表創意度較低，通常更符合事實；1 代表創意度較高)
- `.maxOutputTokens(50)`：在範例中，系統要求 500 個權杖 (3 個權杖約等於 4 個字)，視您希望生成的答案長度而定
- `.topK(20)`：從最多可能出現的字詞中隨機選取一個，用於文字完成功能 (1 到 40)
- `.topP(0.95)`：選取總機率加總為該浮點數 (介於 0 和 1 之間) 的可能字詞
- `.maxRetries(3)`：如果您超出每次要求的配額，可以讓模型重試呼叫 3 次 (例如)

資訊：如要進一步瞭解 temperature、Top-P、Top-K 和 max token 值的確切意義，請參閱[測試文字提示](#)的說明文件頁面。

大型語言模型功能強大，可回答複雜問題，並處理各種有趣的工作。在下一節中，我們將瞭解一項實用工作：從文字中擷取結構化資料。

5. 從非結構化文字中擷取資訊

在上一節中，您產生了一些文字輸出內容。如果您想直接向使用者顯示這項輸出內容，這樣做沒問題。但如果想擷取這項輸出內容中提及的資料，該如何從非結構化文字中擷取這類資訊？

資訊：一般認為結構化資料 (例如資料庫表格、試算表等) 約占資料的 20%，而 80% 的資訊都是非結構化文字資料。

假設您想從某人的簡介或描述中擷取姓名和年齡，您可以調整提示，指示大型語言模型產生 JSON 資料結構，方法如下 (這通常稱為「提示工程」)：

```
Extract the name and age of the person described below.
```

```
Return a JSON document with a "name" and an "age" property,  
following this structure: {"name": "John Doe", "age": 34}  
Return only JSON, without any markdown markup surrounding it.
```

```
Here is the document describing the person:
```

```
---
```

```
Anna is a 23 year old artist based in Brooklyn, New York. She was  
born and raised in the suburbs of Chicago, where she developed a  
love for art at a young age. She attended the School of the Art  
Institute of Chicago, where she studied painting and drawing.  
After graduating, she moved to New York City to pursue her art career.  
Anna's work is inspired by her personal experiences and observations  
of the world around her. She often uses bright colors and bold lines  
to create vibrant and energetic paintings. Her work has been  
exhibited in galleries and museums in New York City and Chicago.
```

```
---
```

```
JSON:
```

良好的提示工程做法：有時，明確要求 LLM 生成特定內容非常重要。我們在此強調只希望傳回 JSON，並顯示要傳回的 JSON 結構範例。我們進一步強調回覆應為 JSON，因此在提示結尾加上 `JSON:`，確保輸出的是 JSON 文字，而非以 Markdown 區塊包圍的 JSON。

修改 `TextPrompts` 類別中的 `model.generate()` 呼叫，將上述整個文字提示傳遞給該呼叫：


```

Response<String> response = model.generate("""
    Extract the name and age of the person described below.
    Return a JSON document with a "name" and an "age" property, \
    following this structure: {"name": "John Doe", "age": 34}
    Return only JSON, without any markdown markup surrounding it.
    Here is the document describing the person:
    ---
    Anna is a 23 year old artist based in Brooklyn, New York. She was born and
    raised in the suburbs of Chicago, where she developed a love for art at a
    young age. She attended the School of the Art Institute of Chicago, where
    she studied painting and drawing. After graduating, she moved to New York
    City to pursue her art career. Anna's work is inspired by her personal
    experiences and observations of the world around her. She often uses bright
    colors and bold lines to create vibrant and energetic paintings. Her work
    has been exhibited in galleries and museums in New York City and Chicago.
    ---
    JSON:
    """)
);

```

如果您在 `TextPrompts` 類別中執行這個提示，應該會傳回下列 JSON 字串，您可以使用 [GSON](#) 程式庫等 JSON 剖析器剖析該字串：

```

$ ./gradlew run -DjavaMainClass=palm.workshop.TextPrompts

> Task :app:run
{"name": "Anna", "age": 23}

BUILD SUCCESSFUL in 24s
2 actionable tasks: 1 executed, 1 up-to-date

```

當然可以！Anna 年滿 23 歲！

資訊：在下一個有關即時通訊語言模型的程式碼研究室中，我們將瞭解 `AiServices` 類別。這個類別提供以註解為導向的方法，可在對話情境中擷取嚴格型別的物件，而不是傳回稍後由 JSON 剖析器剖析的字串。

6. 提示範本和結構化提示

不只是回答問題

PaLM 等大型語言模型功能強大，可回答問題，但用途不僅於此！舉例來說，請在 [Generative AI Studio](#) 中嘗試下列提示 (或修改 `TextPrompts` 類別)。將大寫字詞換成自己的想法，然後檢查輸出內容：

- **翻譯** - 「請將下列句子翻譯成法文：`YOUR_SENTENCE_HERE`」
- **摘要**：「請提供下列文件的摘要：貼上文件」
- **生成創意內容** - 「撰寫有關`TOPIC_OF_THE_POEM`的詩」
- **程式設計** - 「如何以 `PROGRAMMING_LANGUAGE` 撰寫費波那契函式？」

提示範本

如果您嘗試使用上述提示進行翻譯、摘要、生成創意或程式設計工作，您會將預留位置值替換成自己的想法。不過，您也可以利用提示範本定義這些預留位置值，之後再填入資料，不必進行字串處理。

請將 `main()` 方法的整個內容替換為下列程式碼，看看美味又充滿創意的提示：

```
VertexAiLanguageModel model = VertexAiLanguageModel.builder()
    .endpoint("us-central1-aiplatform.googleapis.com:443")
    .project("YOUR_PROJECT_ID")
    .location("us-central1")
    .publisher("google")
    .modelName("text-bison@001")
    .maxOutputTokens(300)
    .build();

PromptTemplate promptTemplate = PromptTemplate.from("""
    Create a recipe for a {{dish}} with the following ingredients: \
    {{ingredients}}, and give it a name.
    """)
);

Map<String, Object> variables = new HashMap<>();
variables.put("dish", "dessert");
variables.put("ingredients", "strawberries, chocolate, whipped cream");

Prompt prompt = promptTemplate.apply(variables);

Response<String> response = model.generate(prompt);

System.out.println(response.content());
```

重要事項：請務必變更專案 ID，使其與您自己的專案名稱相符！

並新增下列匯入項目：

```
import dev.langchain4j.model.input.Prompt;
import dev.langchain4j.model.input.PromptTemplate;

import java.util.HashMap;
import java.util.Map;
```

然後再次執行應用程式。輸出內容應如下所示：

```
$ ./gradlew run -DjavaMainClass=palm.workshop.TextPrompts
```

```
> Task :app:run
```

```
**Strawberry Shortcake**
```

```
Ingredients:
```

```
* 1 pint strawberries, hulled and sliced
* 1/2 cup sugar
* 1/4 cup cornstarch
* 1/4 cup water
* 1 tablespoon lemon juice
* 1/2 cup heavy cream, whipped
* 1/4 cup confectioners' sugar
* 1/4 teaspoon vanilla extract
* 6 graham cracker squares, crushed
```

```
Instructions:
```

```
1. In a medium saucepan, combine the strawberries, sugar, cornstarch, water, and
lemon juice. Bring to a boil over medium heat, stirring constantly. Reduce heat and
simmer for 5 minutes, or until the sauce has thickened.
2. Remove from heat and let cool slightly.
3. In a large bowl, combine the whipped cream, confectioners' sugar, and vanilla
extract. Beat until soft peaks form.
4. To assemble the shortcakes, place a graham cracker square on each of 6 dessert
plates. Top with a scoop of whipped cream, then a spoonful of strawberry sauce.
Repeat layers, ending with a graham cracker square.
5. Serve immediately.
```

```
**Tips:**
```

```
* For a more elegant presentation, you can use fresh strawberries instead of sliced
strawberries.
* If you don't have time to make your own whipped cream, you can use store-bought
whipped cream.
```

美味！

使用提示範本時，您可以在呼叫文字生成方法前，提供必要參數。這是在使用者提供不同值時，傳遞資料及自訂提示的絕佳方式。

如類別名稱所示，`PromptTemplate` 類別會建立範本提示，您可以套用預留位置名稱和值的對應，將值指派給預留位置元素。

結構化提示 (選用)

如要使用更豐富的物件導向方法，也可以使用 `@StructuredPrompt` 註解來建構提示。您可以使用這個註解為類別加上註解，而類別的欄位會對應至提示中定義的預留位置。我們來看看實際運作情況。

首先，我們需要匯入一些新項目：

```
import java.util.Arrays;
import java.util.List;
import dev.langchain4j.model.input.structured.StructuredPrompt;
import dev.langchain4j.model.input.structured.StructuredPromptProcessor;
```

接著，我們可以在 `TextPrompts` 類別中建立內部靜態類別，收集在 `@StructuredPrompt` 註解中描述的提示內傳遞預留位置所需的資料：

```
@StructuredPrompt("Create a recipe of a {{dish}} that can be prepared using only {{ingredients}}")
static class RecipeCreationPrompt {
    String dish;
    List<String> ingredients;
}
```

接著，例項化該新類別，並將食譜的菜餚和食材提供給該類別，然後建立提示並傳遞至 `generate()` 方法，做法與先前相同：

```
RecipeCreationPrompt createRecipePrompt = new RecipeCreationPrompt();
createRecipePrompt.dish = "salad";
createRecipePrompt.ingredients = Arrays.asList("cucumber", "tomato", "feta", "onion", "olives");
Prompt prompt = StructuredPromptProcessor.toPrompt(createRecipePrompt);

Response<String> response = model.generate(prompt);
```

您可以使用 Java 物件，透過 IDE 自動完成欄位，以更安全的方式填補間隙，而不必透過對應填補。

如果您想更輕鬆地將這些變更貼到 `TextPrompts` 類別中，請使用下列完整程式碼：

```

package palm.workshop;

import java.util.Arrays;
import java.util.List;
import dev.langchain4j.model.input.Prompt;
import dev.langchain4j.model.output.Response;
import dev.langchain4j.model.vertexai.VertexAiLanguageModel;
import dev.langchain4j.model.input.structured.StructuredPrompt;
import dev.langchain4j.model.input.structured.StructuredPromptProcessor;

public class TextPrompts {

    @StructuredPrompt("Create a recipe of a {{dish}} that can be prepared using only {{ingredients}}")
    static class RecipeCreationPrompt {
        String dish;
        List<String> ingredients;
    }

    public static void main(String[] args) {
        VertexAiLanguageModel model = VertexAiLanguageModel.builder()
            .endpoint("us-central1-aiplatform.googleapis.com:443")
            .project("YOUR_PROJECT_ID")
            .location("us-central1")
            .publisher("google")
            .modelName("text-bison@001")
            .maxOutputTokens(300)
            .build();

        RecipeCreationPrompt createRecipePrompt = new RecipeCreationPrompt();
        createRecipePrompt.dish = "salad";
        createRecipePrompt.ingredients = Arrays.asList("cucumber", "tomato", "feta", "onion",
"olives");
        Prompt prompt = StructuredPromptProcessor.toPrompt(createRecipePrompt);

        Response<String> response = model.generate(prompt);

        System.out.println(response.content());
    }
}

```

重要事項：請務必變更專案 ID，使其與您自己的專案名稱相符！

步驟 7 · 約 5 分鐘

7. 分類文字和分析情緒

與前一節所學類似，您將探索另一項「提示工程」技術，讓 PaLM 模型分類文字或分析情緒。我們來談談少量樣本提示。只要提供幾個範例，就能讓語言模型更瞭解您的意圖，並朝您期望的方向生成內容。

我們來重新製作 `TextPrompts` 類別，充分運用提示範本：

```

package palm.workshop;

import java.util.Map;

import dev.langchain4j.model.output.Response;
import dev.langchain4j.model.vertexai.VertexAiLanguageModel;
import dev.langchain4j.model.input.Prompt;
import dev.langchain4j.model.input.PromptTemplate;

public class TextPrompts {
    public static void main(String[] args) {
        VertexAiLanguageModel model = VertexAiLanguageModel.builder()
            .endpoint("us-central1-aiplatform.googleapis.com:443")
            .project("YOUR_PROJECT_ID")
            .location("us-central1")
            .publisher("google")
            .modelName("text-bison@001")
            .maxOutputTokens(10)
            .build();

        PromptTemplate promptTemplate = PromptTemplate.from("""
            Analyze the sentiment of the text below. Respond only with one word to describe
            the sentiment.

            INPUT: This is fantastic news!
            OUTPUT: POSITIVE

            INPUT: Pi is roughly equal to 3.14
            OUTPUT: NEUTRAL

            INPUT: I really disliked the pizza. Who would use pineapples as a pizza topping?
            OUTPUT: NEGATIVE

            INPUT: {{text}}
            OUTPUT:
            """);

        Prompt prompt = promptTemplate.apply(
            Map.of("text", "I love strawberries!"));

        Response<String> response = model.generate(prompt);

        System.out.println(response.content());
    }
}

```

重要事項：請務必變更專案 ID，使其與您自己的專案名稱相符！

請注意，提示中提供幾個輸入和輸出內容的範例。這些「少量樣本」可協助 LLM 遵循相同結構。模型收到輸入內容後，會想傳回符合輸入/輸出模式的輸出內容。

執行程式後，應該只會傳回 `POSITIVE` 這個字，因為草莓也很美味！

```
$ ./gradlew run -DjavaMainClass=palm.workshop.TextPrompts

> Task :app:run
POSITIVE
```

情緒分析也是內容分類情境。您可以套用相同的「少樣本提示」方法，將不同文件分類到不同類別。

步驟 8 · 約 0 分鐘

8. 恭喜

恭喜，您已成功使用 LangChain4J 和 PaLM API，以 Java 建構第一個生成式 AI 應用程式！您在過程中發現，大型語言模型功能強大，可處理各種工作，例如問答、資料擷取、摘要、文字分類、情緒分析等。

後續步驟

如要進一步瞭解如何使用 Java 存取 PaLM，請參閱下列程式碼研究室：

- [透過 PaLM 和 LangChain4J，與使用者和文件進行生成式 AI 輔助對話](#)

其他資訊

- [生成式 AI 的常見用途](#)
- [生成式 AI 訓練資源](#)
- [透過 Generative AI Studio 與 PaLM 互動](#)
- [負責任的 AI 技術](#)

參考文件

- [Vertex AI 生成式 AI 說明文件](#)
 - [Github 上的 LangChain4J](#)
-